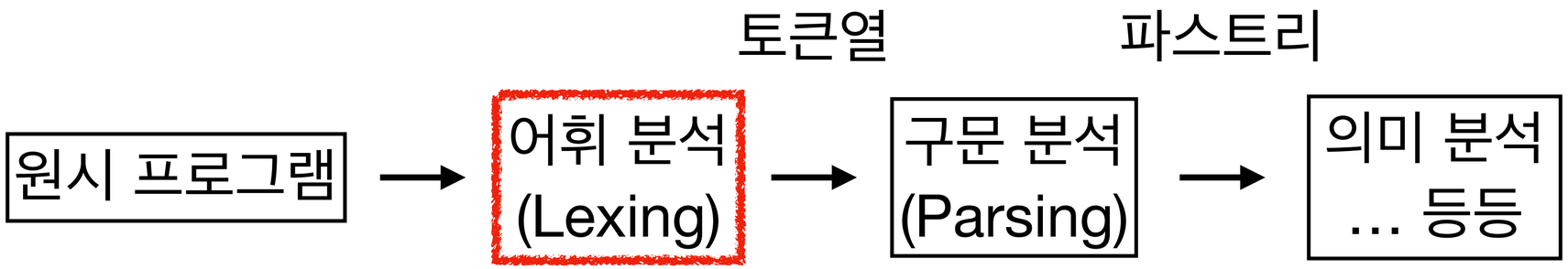
인터프리터 스터디 1

2026 김은빈

컴파일 과정



언어

글자 a와 b만 쓸 수 있는 언어가 있다고 해보자

언어

글자 a와 b만 쓸 수 있는 언어가 있다고 해보자

ε, a, b, ab, ba, aab, bba, abab, aaabb, ...

언어

글자 a와 b만 쓸 수 있는 언어가 있다고 해보자

ε, a, b, ab, ba, aab, bba, abab, aaabb, ...

쓸 수 있는 기호의 집합 T = {a, b}

모든 문자열의 집합 T* (Kleene closure)

집합 안의 요소들로 이어붙여 만들 수 있는 모든 문자열의 집합

언어

T* 에서 조건에 맞는 것만 골라낸 부분집합 = 언어

$L = \{ a^n b^n \mid n \geq 1 \} = \{ ab, aabb, aaabbb, aaaabbbb, \ldots \}$

언어

집합 요소를 다 나열할 수는 없으니..
생성 규칙으로 표현

Noam Chomsky



https://ko.wikipedia.org/wiki/%EC%83%9D%EC%84%B1%EB%AC%B8%EB%B2%95

생성 규칙

예시:

S → ab
S → aSb

a, b = 터미널(terminal, 종단 기호)
문법 규칙의 왼쪽에 절대 나타나지 않는 기호
그냥 글자임

S = 논터미널(non-terminal, 비종단 기호)
문법 규칙의 왼쪽에 나타날 수 있는 기호
왼쪽에 나올 수 있다는 것
= 그 기호를 다른 것으로 치환(교체)할 규칙이 존재한다는 뜻

쓸 수 있는 기호의 집합 T = {a, b}
모든 문자열의 집합 T*
의 T는 터미널이라는 뜻

생성 규칙

$S \rightarrow ab$

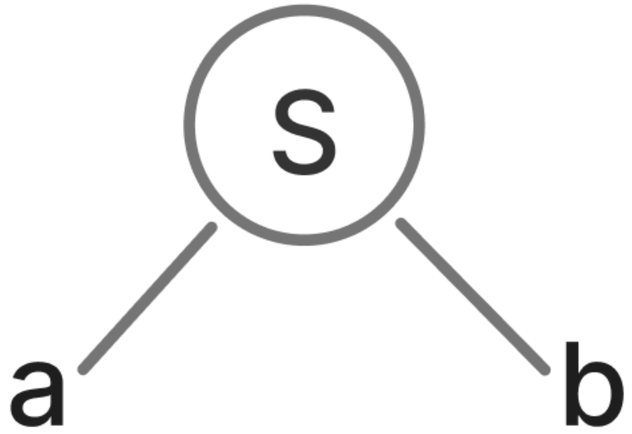
$S \rightarrow aSb$



생성 규칙

$S \rightarrow ab$
$S \rightarrow aSb$

$S \rightarrow ab$



결과: ab

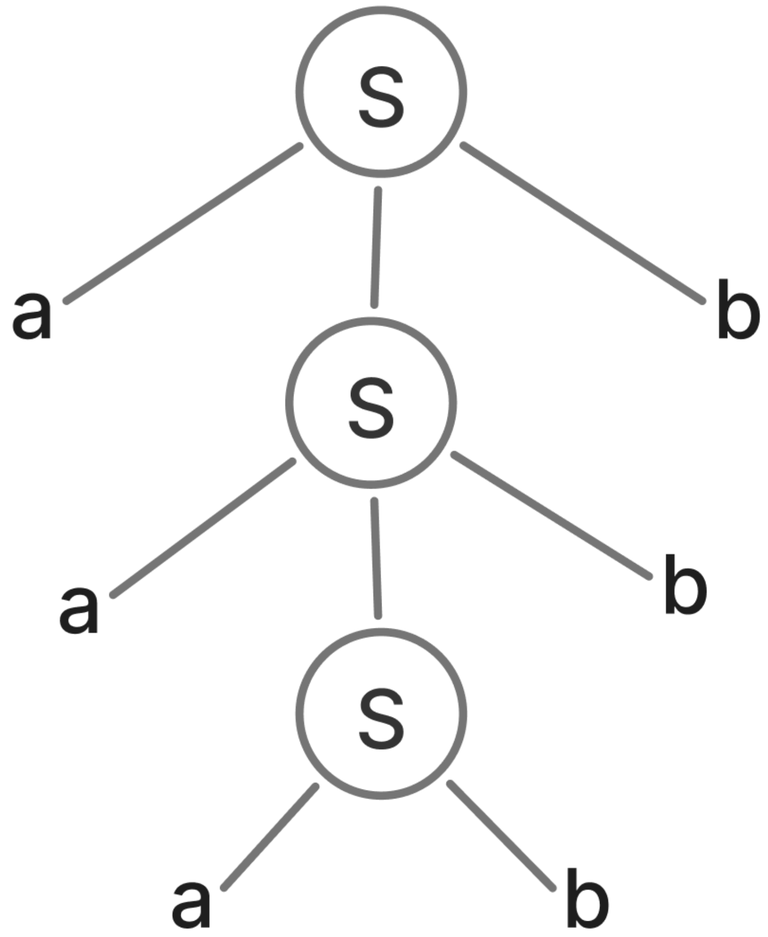
생성 규칙

S → ab
S → aSb

S → aSb

S → aSb

S → ab



결과: aaabbb

생성 규칙

$S \rightarrow ab$

$S \rightarrow aSb$

$S \rightarrow ab$

$S \rightarrow aSb \rightarrow a \cdot ab \cdot b = aabb$

$S \rightarrow aSb \rightarrow aaSbb \rightarrow aa \cdot ab \cdot bb = aaabbb$

문법

문법 G는 아래 4가지로 정의됨

G = (V_N, V_T, P, S)

V_N : 논터미널 목록 위 예에서 {S}

V_T : 터미널 목록 {a, b} (V_N 과 겹치지 않는다)

P : 생성규칙 목록 {S → ab, S → aSb}

S : 시작기호 S

인식기

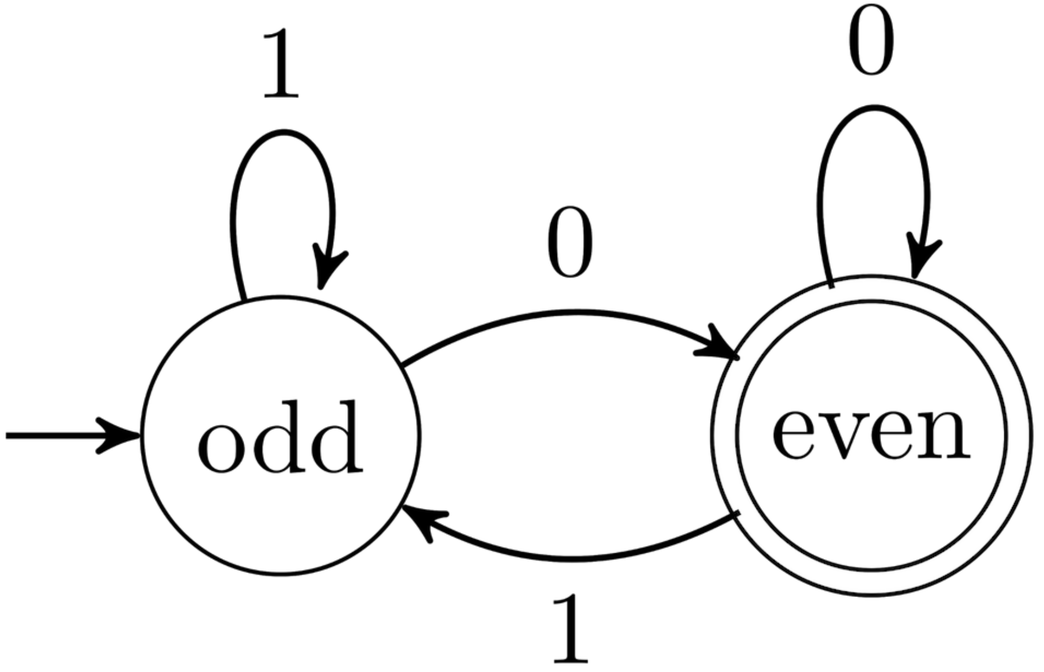
문법에 맞는지 체크하는 기계



https://articlereword.com/grammar-checker/

가장 간단한 구조의 인식기 - 유한 오토마타

현재 상태만 알 수 있는 기계



예시:
0의 갯수가 짝수이면 허용하는 언어
오른쪽처럼 유한 오토마타를 구성할 수 있음

(두겹으로 동그라미 표시한건
문법에 맞는 상태라는 뜻)

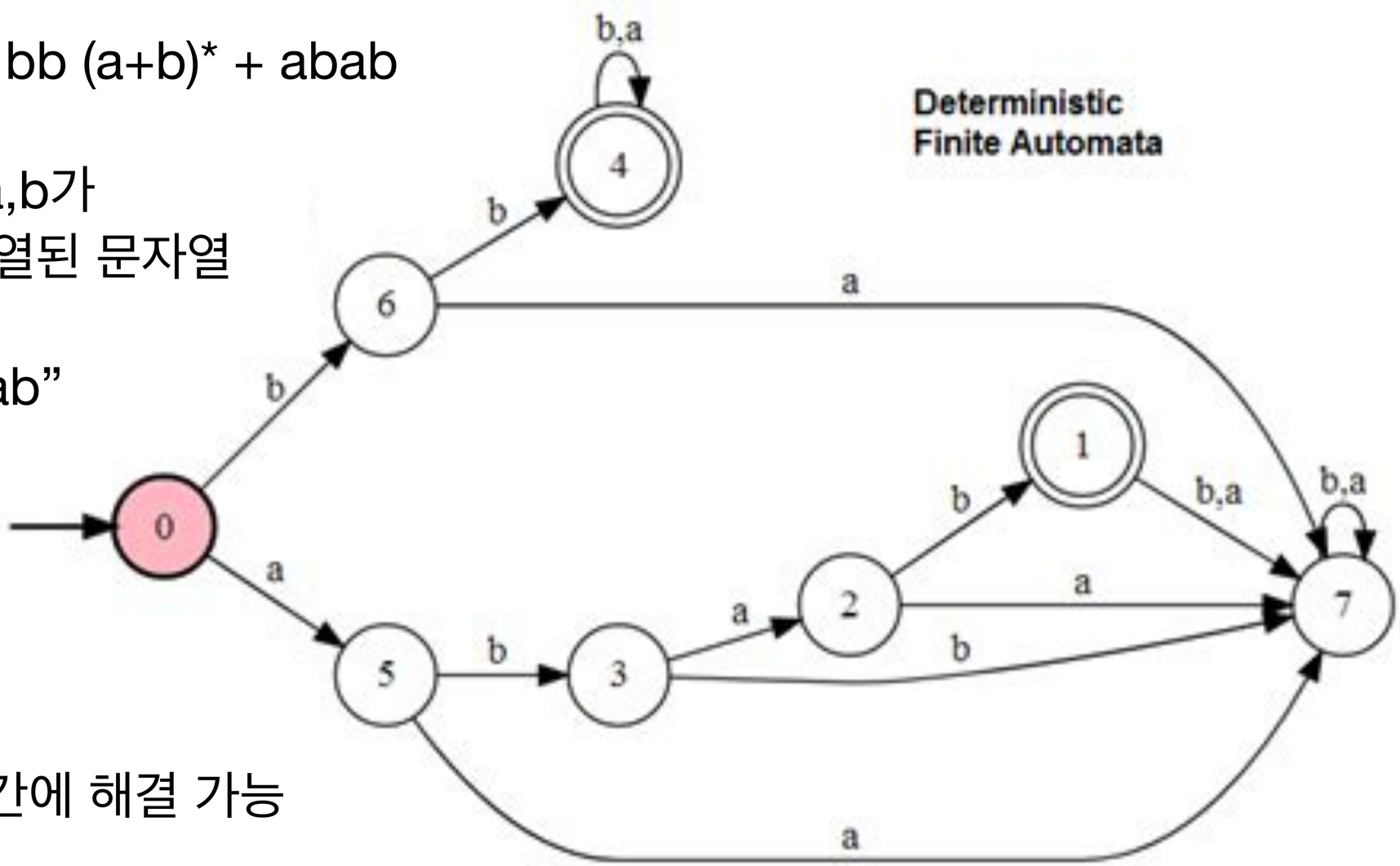
터미널 기호 목록: {0, 1}
홀수 라는 이름의 상태와
짝수라는 이름의 상태를 임의로 정한 것임

https://kidscodecs.com/finite-automata/

가장 간단한 구조의 인식기 - 유한 오토마타

정규표현식: bb (a+b)* + abab

“bb” 뒤에 a,b가
자유롭게 나열된 문자열
or
정확히 “abab”



항상 선형 시간에 해결 가능

특징 1: 유한 오토마타는 정규 언어를 인식할 수 있다.

정규 언어 = 정규표현식이 나타내는 언어
유한 오토마타가 계산할 수 있는 것 = 정규표현식으로 표현할 수 있는 것

정규표현식

∅, ε, a	기본
P + Q (P | Q)	합집합
P · Q (PQ)	연결
P*	Kleene closure: 집합 안의 요소들로 이어붙여 만들 수 있는 모든 문자열의 집합

클레이니 정리(Kleene's theorem)
https://r0k.wiki/posts/regex/regex_02/

특징 1: 유한 오토마타는 정규 언어를 인식할 수 있다.

정규 언어 = 정규표현식이 나타내는 언어
유한 오토마타가 계산할 수 있는 것 = 정규표현식으로 표현할 수 있는 것

정규표현식

∅, ε, a
P + Q (P | Q)
P · Q (PQ)
P*

촘스키 계층

타입	문법	추가되는 제약
0	무제한 (URG)	-
1	문맥의존 (CSG)	$\|\alpha\| \leq \|\beta\|$ (길이가 줄면 안 됨)
2	문맥자유 (CFG)	좌변 = 비단말 하나뿐
3	정규 (RG)	우변 = aB 또는 a 꼴로 고정

https://en.wikipedia.org/wiki/Chomsky_hierarchy

특징 1: 유한 오토마타는 정규 언어를 인식할 수 있다.

정규 언어 = 정규표현식이 나타내는 언어
유한 오토마타가 계산할 수 있는 것 = 정규표현식으로 표현할 수 있는 것

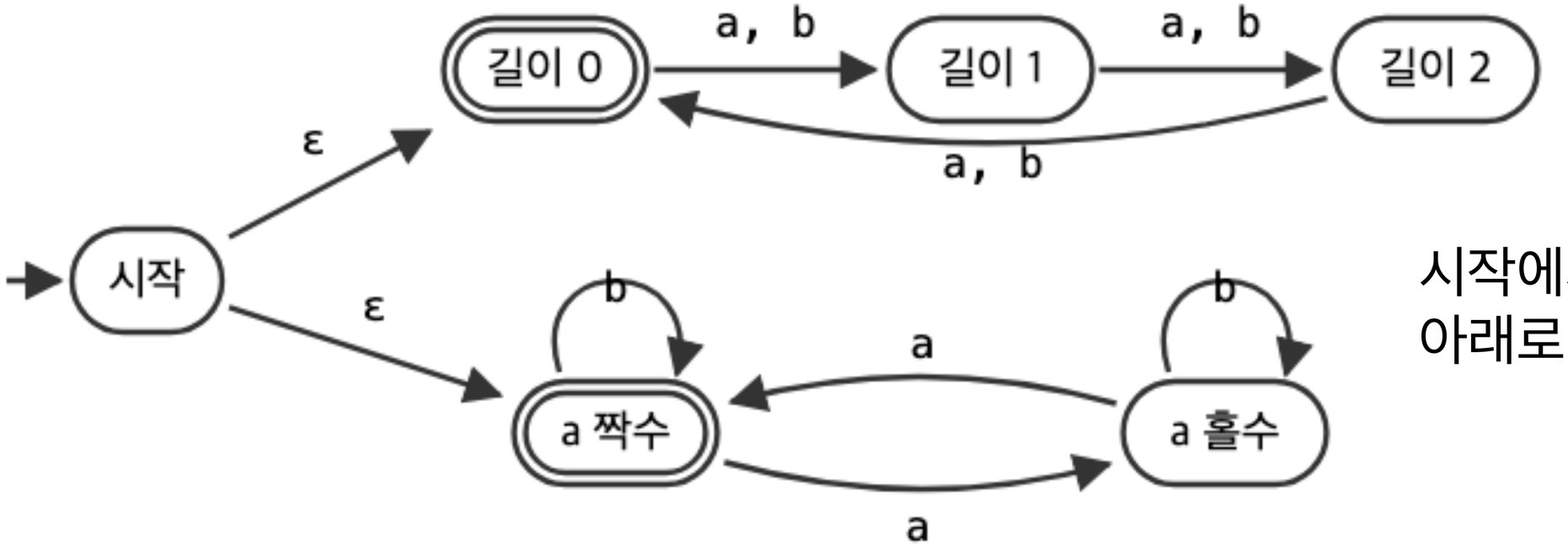
정규표현식

연산	언어 정의	Type 3 규칙 대응
`P + Q`	`L(P) ∪ L(Q)`	`A → P₁ | Q₁` (같은 좌변에 대안 나열)
`P · Q`	`{xy | x∈L(P), y∈L(Q)}`	`A → aB` (단말 소비 후 오른쪽으로 이동)
`P*`	`∪ₙ₌₀^∞ Lⁿ` = `{ε} ∪ L ∪ LL ∪ ...`	`A → aA` (제자리 재귀)

3	정규 (RG)	우변 = `aB` 또는 `a` 꼴로 고정

특징 2: 비결정적 유한 오토마타(NFA), 결정적 유한 오토마타(DFA)로 분류할 수 있다

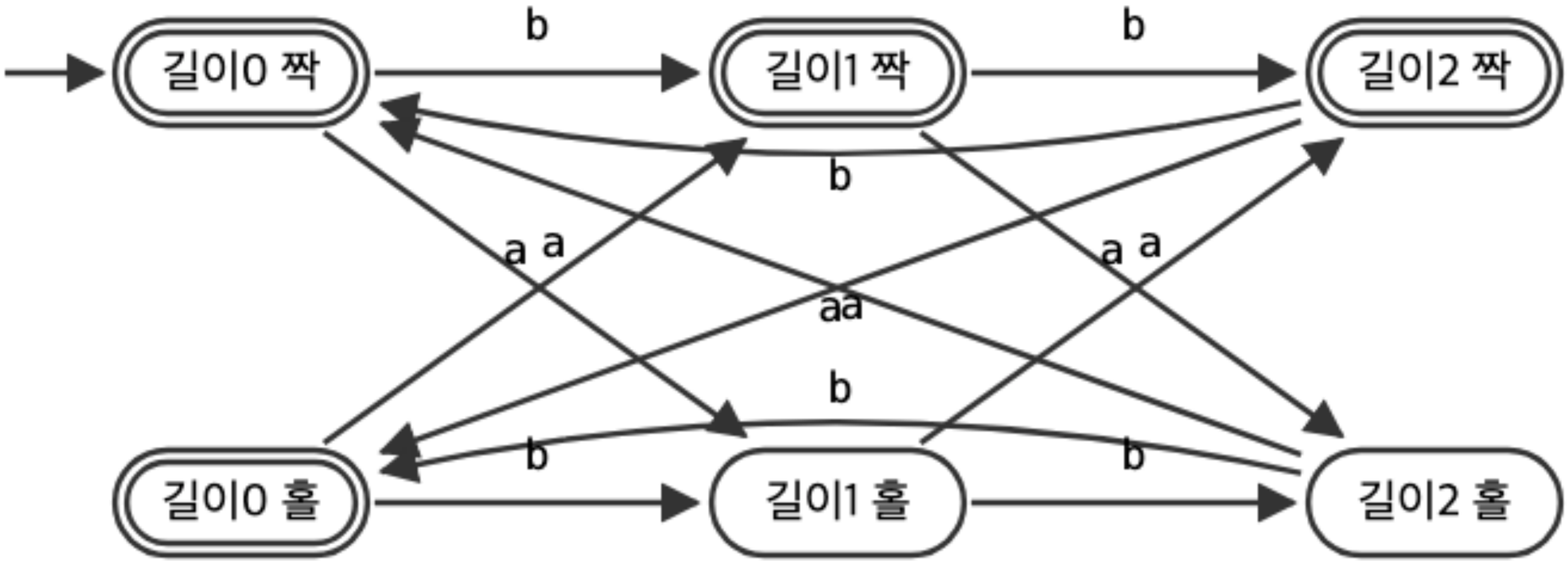
길이가 3 의 배수이거나 a가 짝수 개 NFA 표현



시작에서 위로 가야할지?
아래로 가야할지? 알 수 없음

특징 2: 비결정적 유한 오토마타(NFA), 결정적 유한 오토마타(DFA)로 분류할 수 있다

길이가 3 의 배수이거나 a가 짝수 개 DFA 표현



어휘 분석 lexical analysis 와 유한 오토마타의 연관성

고대의 Lexer, 또는 범용 Lexer (e.g. FLEX, logos)

demo.l

```
%{
#include <stdio.h>
int line = 1;                          /* 그대로 복사되는 C */
%}
LETTER  [a-zA-Z_]                      /* 이름 정의 — 규칙에서 {LETTER} 로 씀 */
DIGIT   [0-9]
%%
"if"                          { printf("IF\n"); }
"else"                        { printf("ELSE\n"); }
{LETTER}({LETTER}|{DIGIT})*   { printf("IDENT   %s\n", yytext); }
{DIGIT}+                      { printf("NUMBER  %s\n", yytext); }
"<="|">="|"=="                { printf("RELOP   %s\n", yytext); }
[-+*/=<>(){};]                { printf("OP      %s\n", yytext); }
"//".*                        { /* 주석: 버림 */ }
\n                            { line++; }
[ \t]+                        { /* 공백: 버림 */ }
.                             { printf("ERROR   line %d: '%s'\n", line, yytext); }
%%
int yywrap(void) { return 1; }
int main(void) { yylex(); return 0; }
```

어휘 분석 lexical analysis 와 유한 오토마타의 연관성

고대의 Lexer, 또는 범용 Lexer (e.g. FLEX, logos)

```
$ flex -o demo.yy.c demo.l && cc -o demo demo.yy.c
$ printf 'if (x1 <= 10) {\n  y = x1 * 2; // 두 배\n} else @\n' | ./demo
IF
OP      (
IDENT   x1
RELOP   <=
NUMBER  10
OP      )
OP      {
IDENT   y
OP      =
IDENT   x1
OP      *
NUMBER  2
OP      ;
OP      }
ELSE
ERROR   line 3: '@'
```

각 정규표현식을 바탕으로 NFA(비결정적 유한 오토마타)를 먼저 구성한 후 DFA(결정적 유한 오토마타)로 바꿔서 렉싱
현대 언어는 언어 특성에 맞게
NFA 안거치고 그냥 DFA 바로 만들도록 각각 구현

FLEX 쓰는 예시: PostgreSQL
postgres/blob/master/src/backend/parser/scan.l

유한 오토마타로 인식할 수 없는 문법도 있음

$$L = \{ a^n b^n \mid n \geq 1 \} = \{ ab, aabb, aaabbb, \cdots \}$$

대응하는 상태가 무한히 나옴

(aaaaaaaaaaaa ⋯)

기억할 공간이 딱 하나만 있어도..

유한 오토마타로 인식할 수 없는 문법도 있음

$$L = \{ a^n b^n \mid n \geq 1 \} = \{ ab, aabb, aaabbb, \cdots \}$$

대응하는 상태가 무한히 나옴

(aaaaaaaaaaaa ⋯)

기억할 공간이 딱 하나만 있어도..

유한 오토마타로 인식할 수 없는 문법도 있음

$$L = \{ a^n b^n \mid n \geq 1 \} = \{ ab, aabb, aaabbb, \cdots \}$$

괄호 짝이 맞는지 판단하는 것도

유한 오토마타로 못 하는 일

푸시다운 오토마타

유한 오토마타 + 스택으로 상태 기억 = 푸시다운 오토마타

푸시다운 오토마타는 CFG를 인식할 수 있다

밑바닥부터 만드는
인터프리터 p. 36

타입	문법	추가되는 제약
0	무제한 (URG)	-
1	문맥의존 (CSG)	`|α| ≤ |β|` (길이가 줄면 안 됨)
2	문맥자유 (CFG)	좌변 = 비단말 하나뿐
3	정규 (RG)	우변 = `aB` 또는 `a` 꼴로 고정

스택을 pushdown store라 부르기도 한다고..

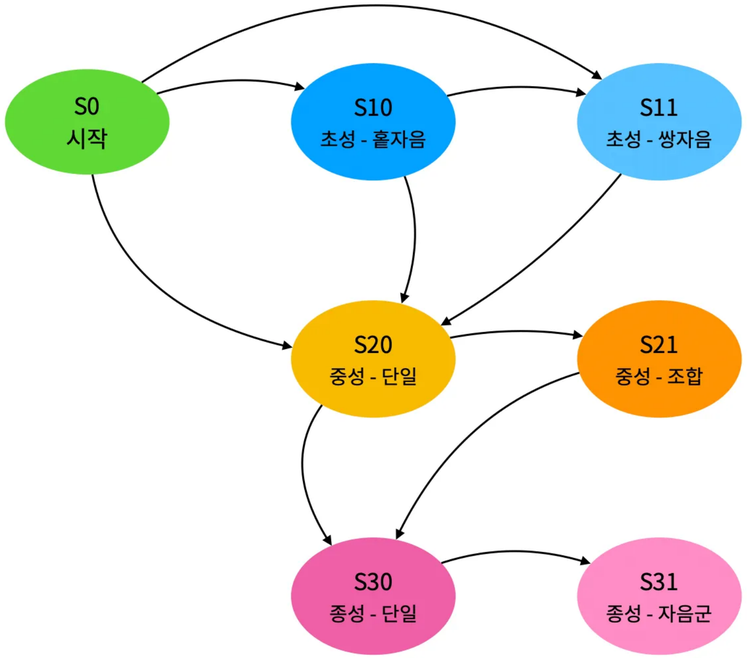
재밌는거

한국어의 생성규칙[1]

- (시작 변수) → (주어) (서술어). | ...
- (주어) → (명사구)(주격 조사) | ...
- (서술어) → (형용사) | ...
- (명사구) → (명사구)(관형격 조사) (명사구) | (명사) | ...
- (명사) → 고양이 | 발 | ...
- (주격 조사) → 이 | 가
- (관형격 조사) → 의
- (형용사) → 부드럽다 | ...
- ...

문자열 생성 예시[2]

(시작 변수)
⇒ **(주어) (서술어).**
⇒ **(명사구)(주격 조사)** (서술어).
⇒ **(명사구)(관형격 조사) (명사구)**(주격 조사) (서술어).
⇒ **(명사)**(관형격 조사) (명사구)(주격 조사) (서술어).
⇒ **고양이**(관형격 조사) (명사구)(주격 조사) (서술어).
⇒ 고양이**의** (명사구)(주격 조사) (서술어).
⇒ 고양이의 **(명사)**(주격 조사) (서술어).
⇒ 고양이의 **발**(주격 조사) (서술어).
⇒ 고양이의 발**이** (서술어).
⇒ 고양이의 발이 **(형용사)**.
⇒ 고양이의 발이 **부드럽다**.



namu.wiki/w/오토마타

https://jay-wkjun.github.io/2024/12/17/Korean-in-Browser.html

재밌는거

사실 아직 이해 안되는 부분..

키위

/NNP

형태소 분석

아호-코라식

14개 언어

문서 토론 읽기 편집 역사 보기

위키백과, 우리 모두의 백과사전.

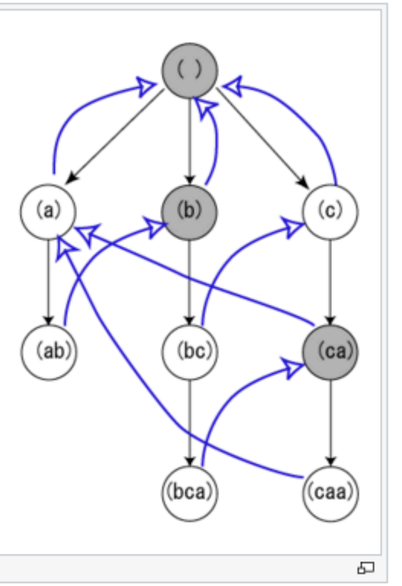
(아호 코라식 알고리즘에서 넘어옴)

아호-코라식(Aho-Corasick)은 문자열 검색을 위하여 Alfred V. Aho와 Margaret J. Corasick이 고안한 자료 구조이다.

패턴 1개를 탐색하는 매칭 알고리즘은 선형 시간에 구현됨을 KMP 등 여러 알고리즘을 통하여 증명되었다. 하지만 패턴 집합에 대하여 이러한 알고리즘을 수행하게 되면 패턴 개수에 비례하여 그 속도가 느려지게 된다. 즉, $O(m+zn)$의 시간 복잡도를 가지게 된다.(m: 모든 패턴들의 길이 합, z: 패턴 수, n: text 크기) 하지만 아호-코라식을 이용하면 $O(m+n+k)$의 시간 복잡도로 패턴 집합에 대하여 패턴 길이와 텍스트의 선형 시간에 탐색을 처리할 수 있게 된다.(k: 텍스트 내에 패턴의 발생 수) 아호-코라식을 구현하기 위하여 Keyword Tree, Failure link, Output link 자료 구조를 사용한다.

키워드 트리 [편집]

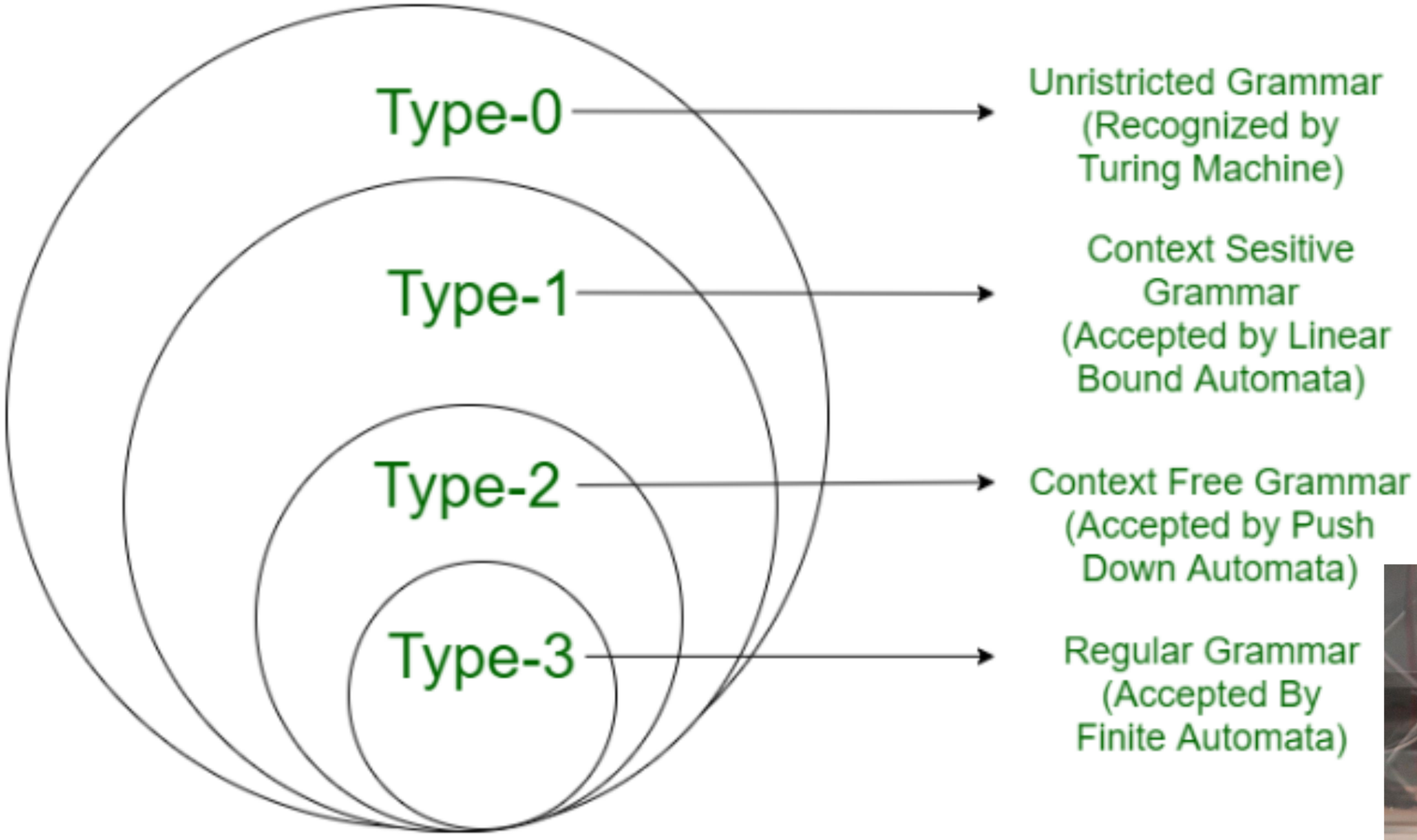
키워드 트리(Keyword Tree)는 패턴의 글자 하나를 하나의 간선으로 한다. 그리고 서로 패턴들의 접두사가 최대한 같을 때까지는 같은 정점으로 간선을 따라가고 글자가 달라지면 다른 간선으로 가도록 만들게 된다. 전 처리 과정에서 Keyword Tree를 만들면서 모든 패턴들의 길이의 합의 시간 복잡도 안에 만들 수 있게 된다.



https://github.com/bab2min/Kiwi

https://accesson.kr/kjdh/v.1/1/109/43508 https://aclanthology.org/J97-2003.pdf

부록

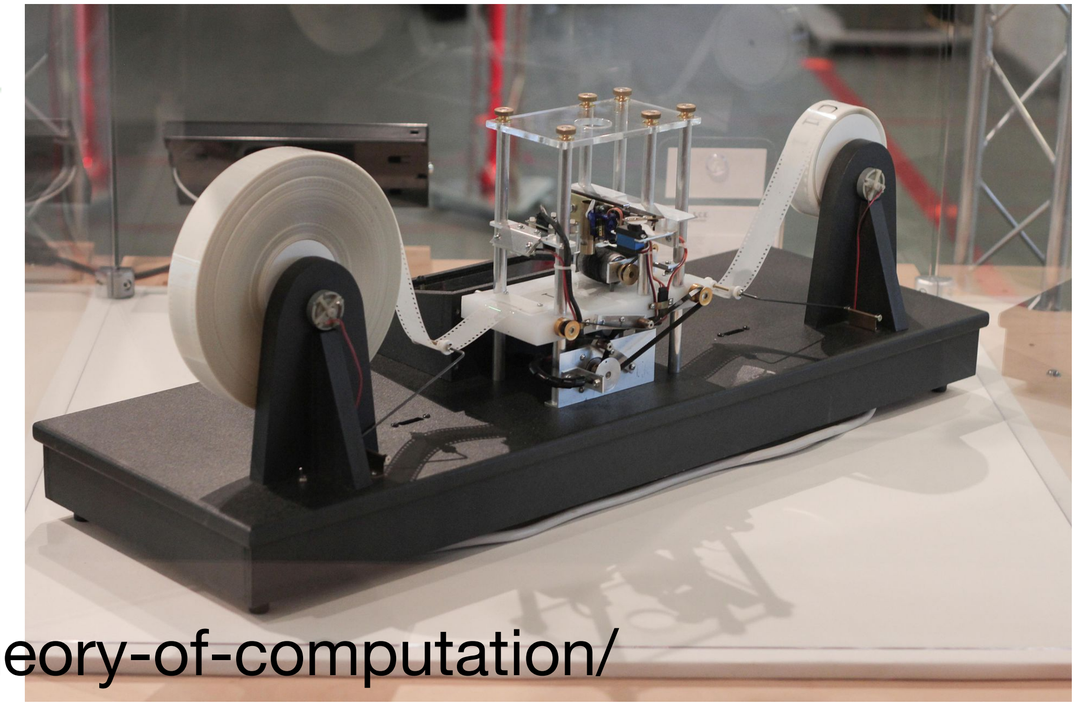


Unristricted Grammar
(Recognized by
Turing Machine)

Context Sesitive
Grammar
(Accepted by Linear
Bound Automata)

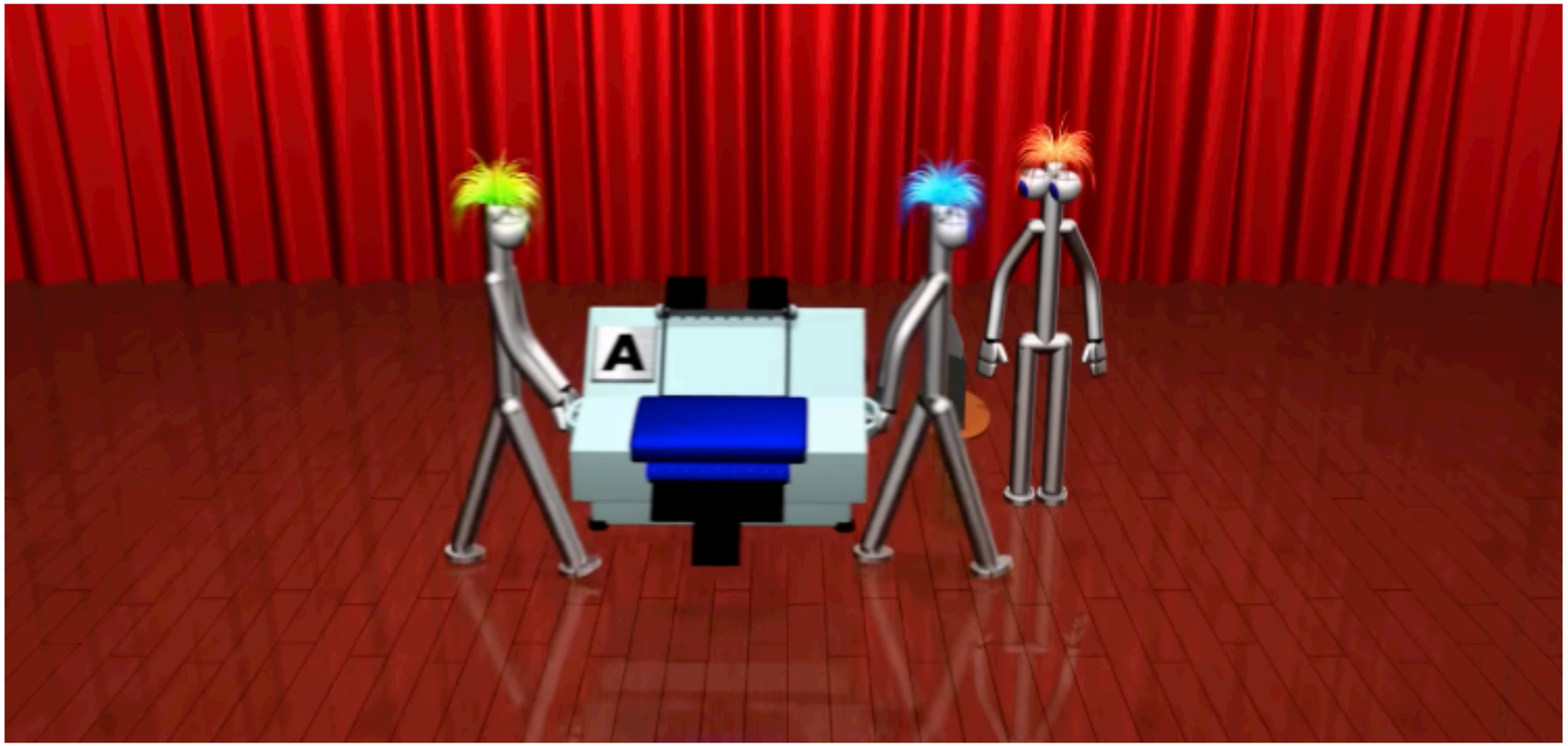
Context Free Grammar
(Accepted by Push
Down Automata)

Regular Grammar
(Accepted By
Finite Automata)



https://www.geeksforgeeks.org
/theory-of-computation/chomsky-hierarchy-in-theory-of-computation/

Halting Problem (정지 문제)



https://youtu.be/92WHN-pAFCs

https://www.udiprod.com/halting-problem/#faq